Operators

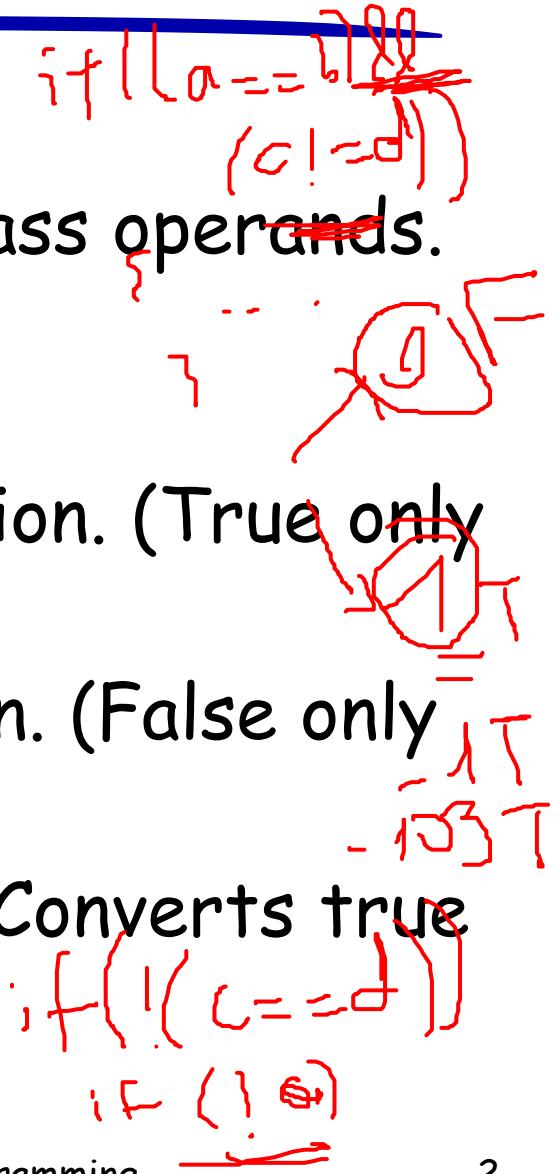
- Arithmetic operators (+,-,*,/,%)
  - General meanings are obvious.
  - What is important is the following: If one of the operands is of a wider type, the result is also of that type. (Its importance will be more obvious soon.)
    - Eg: Result of int+float is float. Result of float+double is double.
  - In C language, there are two types of division: integer division and float division.
    - If both operands are of integer class, we perform integer division and the result is obtained by truncating the decimal part.
      - Eg: `8/3` is `2`, not `2.666667`.
    - If one of the operands is of float class, the result is float.
      - Eg: `8.0/3` or `8/3.0` or `8.0/3.0` is `2.666667`, not `2`.

Operators

- Remainder operator is `%`. Both operands must be of integer class.
  - Eg: `10%6` is `4` (equivalent to `10 mod 6`)
- +,-,*,/,% have left-to-right associativity. That means `a/b/c` is equivalent to `(a/b)/c`, but not `a/(b/c)`.

Operators

- Logic operators (&&, ||, !)
  - Logic operators take integer class operands.
    - Zero means false.
    - Anything non-zero means true.
  - "&&" does a logical-AND operation. (True only if both operands are true.)
  - "||" does a logical-OR operation. (False only if both operands are false.)
  - "!" does a negation operation. (Converts true to false, and false to true.)



Operators

- Logic operators follow the logic rules

a	b	a && b	a || b
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

- The order of evaluation is from left to right
- As usual parenthesis overrides default order

Operators

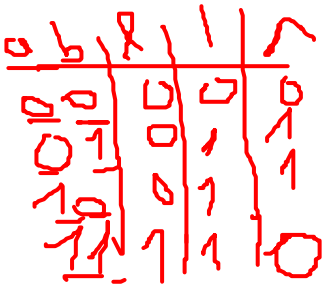
- If the first operand of the "`&&`" operator is false, the second operand is not evaluated at all (since it is obvious that the whole expression is false).
  - Eg: In the expression below, if the values of `b` and `c` are initially `0` and `1`, respectively,

    ```
    a = b && (c=2)
    ```

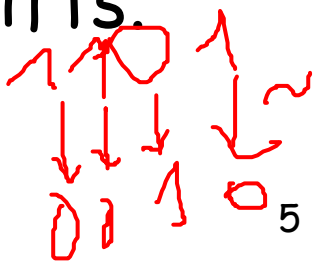
    then the second operand is not evaluated at all, so `c` keeps its value as `1`.
- Similarly, if the first operand of the "`||`" operator is true, the second operand is not evaluated at all.

Operators

- Bitwise operators (&, |, ^, <<, >>, ~)
  - Bitwise operators take integer class operands.
    - For the logic operators, the variable represents a single logical value, true or false.
    - For the bitwise operators, each bit of the variable represents true or false.
  - &, |, and ^ perform bitwise-AND, -OR, -XOR, respectively.
  - << and >> perform left- and right-shifts.
  - "~" takes bitwise one's complement.







Operators

Operation	Result
5 & 10 (0000 0101 & 0000 1010)	0 (0000 0000)
5 && 10 (0000 0101 && 0000 1010)	1 (0000 0001)
5 | 10 (0000 0101 | 0000 1010)	15 (0000 1111)
8 ^ 10 (0000 0111 ^ 0000 1010)	13 (0000 1101)
7 << 2 (0000 0111 << 0000 0010)	28 (0001 1100)
7 >> 2 (0000 0111 >> 0000 0010)	1 (0000 0001)
~5 (~0000 0101)	-6 (in two's complement) (1111 1010)

Operators

- Other assignment operators (+=, -=, *=, /=, %=)
  - Instead of writing `a=a+b`, you can write `a+=b` in short. Similar with `-=`, `*=`, `/=`, and others.

Operators

- Pre/Post increment/decrement operators (++, --)
  - The operator ++ increments the value of the operand by 1.
    - If the operator comes BEFORE the variable name, the value of the variable is incremented before being used, i.e., the value of the expression is the incremented value. This is pre-increment.
    - In post-increment, the operator is used after the variable name, and incrementation is performed after the value is used, i.e., the value of the expression is the value of the variable before incrementation.

Operators

- Eg:

```
a=10;          c=10,
b=++a;         d=c++;
```

Both **a** and **c** will become **11**, but **b** will be **11** while **d** is **10**.

Operators

- Comparison operators (==,!=,<,<=,...)
  - "==" is the "is equal to" operator. Like all other comparison operators, it evaluates to a Boolean value of true or false, no matter what the operand types are.
  - IMPORTANT: When you compare two float values that are supposed to be equal mathematically, the comparison may fail due to the loss of precision discussed before.

Operators

- Floating Point Comparison
  - To compare two floating point or double values, we have to consider the precision into the comparison. For example, if two numbers are 3.1428 and 3.1415, then they are same up to the precision 0.01, but after that, like 0.001 they are not same.
  - IMPORTANT: To compare using this criteria, we will find the absolute value after subtracting one floating point number from another, then check whether the result is lesser than the precision value or not. By this we can decide that they are equivalent or not.

Operators

- Floating Point Comparison

```
#include <math.h>
...
//a = 2.03547415;
//b = 2.03547428;
float epsilon = 0.01f;
if(fabs(x - y) < epsilon)
        //a=2.03 is equivalent to b=2.03
        printf("They are equivalent");
else
        printf("They are NOT equivalent");

double epsilon = 0.0000001f;
if(fabs(x - y) < epsilon)
        printf("They are equivalent");
else
        //a = 2.03547415 is NOT equivalent to b=2.03547428
        //Last digits in fractional part are different
        printf("They are NOT equivalent");
```

Operators

Symbol	Usage	Meaning
==	x == y	is x equal to y?
!=	x != y	is x not equal to y?

>	x > y	is x greater than y?
<	x < y	is x less than y?
>=	x >= y	is x greater than or equal to y?
<=	x <=y	is x less than or equal to y?

Operators

- We can create complex expressions by joining several expressions with logic operators.

Symbol	Usage	Meaning
&&	exp1 && exp2	AND
||	exp1 || exp2	OR
!	! exp	NOT

Operators

- While using multiple operators in the same expression, you should be careful with the precedence and associativity of the operands.
  - Eg: The following does NOT check if **a** is between **5** and **10**.

```
bool = 5<a<10;
```

    - **bool** will be true if **a** is **20**. (Why?)
  - Don't hesitate to use parentheses when you are not sure about the precedence (or to make things explicit).

Operator precedence table

Operator	Associativity
() [] . ->	left-to-right
++ -- + - ! ~ (type) * & sizeof	right-to-left
* / %	left-to-right
+ -	left-to-right
<< >>	left-to-right
< <= > >=	left-to-right
== !=	left-to-right
&	left-to-right
^	left-to-right
|	left-to-right
&&	left-to-right
||	left-to-right
?:	right-to-left
= += -= *= /= %= &= ^= |= <<= >>=	right-to-left
,	left-to-right

Operators

- Precedence, associativity, and order of evaluation:
  - In the table given in the previous slide, precedence decreases as you go down.
  - If two operands in an expression have the same precedence, you decide according to the associativity column.
  - There is a common misunderstanding about associativity.
    - **Note that associativity has nothing to do with the order of evaluation of the operands.**
    - **Order of evaluation of operands is not specified in C language.**
  - **It is strongly recommended that you read and understand Section 2.12 (pp. 52-54) in the book by Kernighan & Ritchie.**

Type casting

- Also called *coersion* or *type conversion*.
- It does NOT change the type of a variable. It is not possible to change the type of a variable.
- What casting does is to convert the type of a value.

Type casting

- Eg:
```
int a=10, b=3;
float f, g;
f=a/b;
g=(float)a/b;
```

- The type of `a` does not change; it is still and integer. However, in the expression `(float)a/b`, the value of `a`, which is `10`, is converted to float value of `10.0`, and then it is divided by `b`, which is `3`. Thus, we perform float division and `g` becomes `3.3333...`
- On the other hand, we perform an integer division for `f`, so it becomes `3`.